

Audio Illusions With a Teensy4.0

[Github](#)

Authors: Samuel Miller and Greer Page

Date: 5/5/2026

CMPE 3815 Microcontroller Systems

University of Vermont

Introduction

The purpose of this report is to detail the process of using a Teensy4.0 microcontroller to generate various audio illusions. Audio illusions are sounds that the brain perceives but do not exist in a physical sense. This project aims to demonstrate a variety of well known auditory illusions in an attempt to explore the Teensy's audio output capabilities.

Illusion Overview

Binaural Beats

The binaural beats illusion is created using two tones with slightly different frequencies. The brain initially sees these two tones as separate, leading to the "creation" of a new third tone. This third tone's frequency is equal to the difference of the two initial frequencies. For example, if 440 Hz and 444 Hz tones are played, the resulting tone will be 4 Hz. In this way, the frequency of this tone can be controlled through the output of the initial tones.

The creation of this third tone is said to have possibly real applications in psychological fields, promoting brainwave entrainment when used for prolonged periods of time. The effects of this illusion can be improved sleep, aided anxiety management, and improved focus (Ingendoh).

Missing Fundamental

The second audio illusion, the missing fundamental, occurs when two harmonically related frequencies are played while an absent lower pitched tone is perceived by the brain. Harmonics are N multiples of the same frequencies, for examples 110 Hz, 220 Hz, and 330 Hz. Using these frequencies, a missing fundamental can be created by playing 220 Hz, 330 Hz, and 440 Hz, resulting in a 110 Hz frequency being constructed by the brain's auditory systems. This occurs because the brain is trained to find patterns in the sound it is perceiving, leading to the creation of the missing fundamental.

Once again, this auditory illusion does have real world applications. In general, smaller speakers are unable to produce lower frequencies to produce bass. In order to work around this, harmonics of the bass notes are used to create the illusion of hearing the missing lower frequency notes. By using this technique, mixes of songs can be enjoyed universally, and helps to greatly enhance lower quality systems (Campo).

Shepard Tone

The Shepard tone is the final audio illusion the system creates, and is based upon the creation of a pitch that rises endlessly. The phenomenon is commonly compared to a barber pole, seeming to be constantly rising. This illusion was created by a psychologist named Roger Shepard in 1964. In order to compose this sound, sine waves are layered together each separated by one octave. All sine waves increase in frequency at the same rate, and as a tone reaches the higher limit it begins to fade out. At the same time, a new frequency at the lower limit fades in, continuing the illusion of a constant rising tone. The tone is commonly used to build tension in movies or video games with its ever-increasing frequency (Splice).

General Approach

The Teensy is programmable from the Arduino IDE by installing the Teensy board extension. It uses the same sketch layout as an Arduino, allowing for an easy transition from working with the Arduino UNO. Several powerful built-in libraries are also installed with this extension.

To create audio from the Teensy, the built-in Audio library can be implemented. This library uses high speed I2S communication to generate, mix, filter, and output audio signals, along with a myriad of other functions. The library works well in tandem with the Teensy audio-shield. This device provides convenient digital to analog conversion and audio output.

To implement multiple illusions of entirely different origins, a modular-application is built. This means that several files are used to compartmentalize the code, rather than just writing the entire application in one Arduino sketch. The file tree used to implement this approach can be seen in Figure 1.

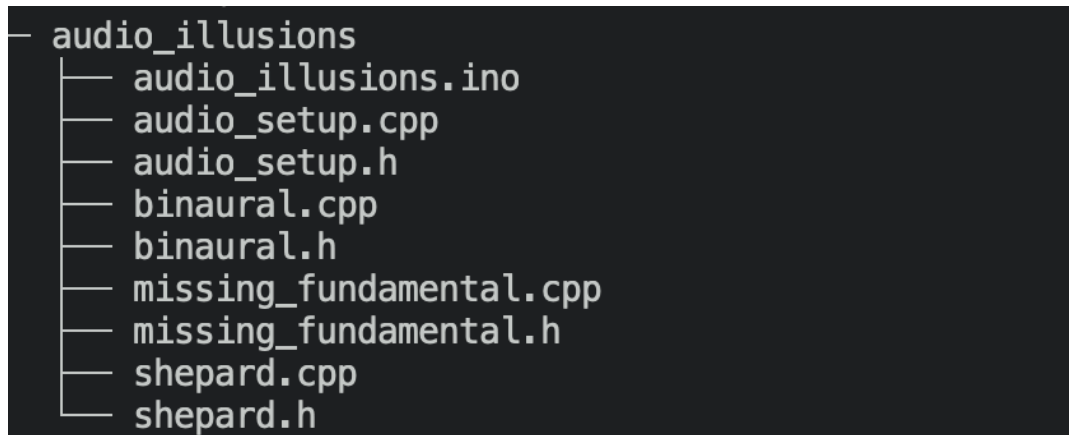


Figure 1: File tree of the modular application used for this project.

Figure 1 shows that each illusion has both a *.cpp* and *.h* file associated with it. The *.cpp* files contain the logic for generating and controlling its respective illusions. The *.h* files are header files. These are used to tell the compiler which functions should be used across multiple files, conveniently linking the logic stored in separate files.

Using this system, the main *audio_illusions.ino* file is used as the main control, switching which illusion is actively being outputted from the device. Then, *audio_setup* configures the universal audio output across the application for each individual illusion to play through. Then, each illusion is programmed in its respective file, with a function to play the illusion in the loop of the main sketch and functions to set up and stop the illusions.

Global Audio Setup

The Audio library can only have one instance of the headphone output initialized at a time. This means that one instance of this object must be shared between all files in the program. This is achieved by using mixers. Mixers are a built-in object in Audio that allow several oscillators (sound waves) to be played together and mixed. The mixing process involves controlling gains of each mixer input. For the purpose of this project, there is a master mixer declared in *audio_setup* that routes directly into the headphone output. The code for this setup is found in Figure 2.

```

AudioMixer4 masterMixer; // create master mixer for all illusions to pass through
AudioOutputI2S audioOut; // create single audio output from headphone jack
AudioControlSGTL5000 sgtl5000; // create audio controller object

// output the master mix from the headphone jack
AudioConnection masterLeft(masterMixer, 0, audioOut, 0); // left speaker
AudioConnection masterRight(masterMixer, 0, audioOut, 1); // right speaker

void setupAudioSystem() {
  /* set up audio system */

  AudioMemory(60); // allocate memory to audio
  sgtl5000.enable(); // start controller

  // set gain for each channel of master mixer (1 for max gain)
  masterMixer.gain(0, 1.0);
  masterMixer.gain(1, 1.0);
  masterMixer.gain(2, 1.0);
  masterMixer.gain(3, 1.0);
}

```

Figure 2: Code to initialize global audio output. *masterRight* and *masterLeft* are arbitrary patch cord variable names that serve to connect audio components.

As well as the final output stages, *audio_setup* implements master volume control by reading a potentiometer and scaling its value to a comfortable volume range. The function handling volume control is found in Figure 3.

```

void volumeControl() {
  /* Control device output volume */
  int pot = analogRead(VOLUME_PIN); // read volume pot
  float volume = pot / 2046.0; // keep volume from 0 - 0.5 (comfortable range)
  if (volume < 0.002) sgtl5000.volume(0); // make sure it goes to 0
  else sgtl5000.volume(volume); // set volume
}

```

Figure 3: Function of control master volume of the device.

As the Teensy has 10 bit analog inputs, it reads potentiometers from 0 - 1023, so dividing the potentiometer value by 2046 keeps the value between 0 and 0.5, which is determined to be a comfortable volume range. The connection scheme for volume control can be found in Table 1.

Teensy Pin	Connection	Function
A0	Potentiometer wiper	Volume control

Table 1: Connection scheme for volume control.

Illusion Implementation

Binaural Beats

The first illusion to implement is the binaural beats, where a perceived beating sound is heard from two oscillators of nearly the same frequency. Implementing this illusion is very simple. Initially the goal is to get a static example of this illusion. To do so, two oscillators of similar frequencies are created, mixed together, and sent through the final mixer. See Figure 4.

```
// Declaration of sine wave objects
AudioSynthWaveformSine base;
AudioSynthWaveformSine beat;

// Declaration of audio mixer
AudioMixer4 binMixer;

// Routing sine waves to the mixer
AudioConnection bb1(base, 0, binMixer, 0);
AudioConnection bb2(beat, 0, binMixer, 1);

// Routing local mixer to master mixer
AudioConnection bb3(binMixer, 0, masterMixer, 2);

void setupBinaural() {
  /* To run in setup() in audio_illusions.ino */
  // set gain as even for both.
  binMixer.gain(0, 0.5);
  binMixer.gain(1, 0.5);
}
```

Figure 4: Initialization and mixing of two oscillators. This mix is run to channel 2 of the master mixer.

Once it's verified that the correct beating frequency can be heard, it is time to implement control over the illusion. To do so, a potentiometer controlling the tone of the base frequency and a potentiometer controlling the beating tone are implemented. The base tone is set to be frequencies of the notes A2 to A4 depending on the potentiometer's setting. The code to define the frequency can be found in Figure 5.

```

float getNoteFrequency(int potValue) {
    /* Get a note frequency based on a potentiometer */

    // divide 1024 into 24 equal buckets (one per note)
    int noteIndex = potValue / (1024 / 24);
    float frequency;

    switch (noteIndex) {
        // get note frequency
        case 0: frequency = 110.00; break; // A2
        case 1: frequency = 116.54; break; // A#2
        case 2: frequency = 123.47; break; // B2
        case 3: frequency = 130.81; break; // C3
        case 4: frequency = 138.59; break; // C#3
        case 5: frequency = 146.83; break; // D3
        case 6: frequency = 155.56; break; // D#3
        case 7: frequency = 164.81; break; // E3
        case 8: frequency = 174.61; break; // F3
        case 9: frequency = 185.00; break; // F#3
        case 10: frequency = 196.00; break; // G3
        case 11: frequency = 207.65; break; // G#3
        case 12: frequency = 220.00; break; // A3
        case 13: frequency = 233.08; break; // A#3
        case 14: frequency = 246.94; break; // B3
        case 15: frequency = 261.63; break; // C4
        case 16: frequency = 277.18; break; // C#4
        case 17: frequency = 293.66; break; // D4
        case 18: frequency = 311.13; break; // D#4
        case 19: frequency = 329.63; break; // E4
        case 20: frequency = 349.23; break; // F4
        case 21: frequency = 369.99; break; // F#4
        case 22: frequency = 392.00; break; // G4
        case 23: frequency = 440.00; break; // A4
        default: frequency = 220.00; break; // default to A3
    }
    return frequency;
}

```

Figure 5: Function to take potentiometer reading and convert it to a note's frequency.

With a dynamic base tone implemented the beats must be controlled. This is achieved by mapping the reading of the beat pin from 1 to 10. With this, at the potentiometer's highest reading, the frequency of the beat oscillator will be 10Hz greater than the frequency of the base. This manifests as a 10Hz beating tone playing from the device. To lower the beat rate, simply adjust the potentiometer.

The connection scheme for the control of this illusion can be found in Table 2.

Teensy Pin	Connection	Function
A1	Potentiometer wiper	Set base tone
A2	Potentiometer wiper	Set beat rate

Table 2: Connection scheme for base tone/beat rate.

Missing Fundamental

The initial approach taken towards demonstrating the missing fundamental illusion is to use a potentiometer to cycle between playing a sine wave of the fundamental frequency and sine waves of several upper harmonics of the fundamental. This yields sub-optimal results. While the fundamental can be heard while the harmonics play, it is not heard clearly through the harmonics. It often sounds an octave or two too high, which makes sense as this is what is physically coming from the speakers.

This discrepancy is likely due to the fact that the overtones of a tone have very specific amplitudes and wavelengths, affecting human perception of how they interact in the ear to hint towards the fundamental. Because of this, filtering an overtone-heavy signal to remove the fundamental frequency seemed like a better approach to demonstrate this illusion as these harmonics will be naturally spaced and balanced.

Fortunately, Audio has a built-in highpass filter object. This can be initialized and routed similarly to waves or mixers, as shown in Figure 6.

```
AudioSynthWaveform fundamental; // create the wave object to be filtered
AudioFilterBiquad highpass; // filter to remove the fundamental
AudioMixer4 mainMixer; // create a mixer to connect differnt frequencies

// Routing
AudioConnection cable1(fundamental, 0, mainMixer, 0); // normal tone to mixer channel 0
AudioConnection cable2(fundamental, 0, highpass, 0); // normal tome to filtered tone
AudioConnection cable3(highpass, 0, mainMixer, 1); // filtered tone to mixer channel 1
AudioConnection cable4(mainMixer, 0, masterMixer, 1); // patch to master
```

Figure 6: Initialization of audio objects and routing for the missing fundamental demonstration.

Using the highpass filter, the fundamental frequency can be filtered, leaving only the overtones playing. With this setup, the fundamental rings clear even when it's filtered. It should be noted that the *fundamental* object is defined as an *AudioSynthWaveForm*, rather than a sine wave. This

is so it can be defined as a triangular wave as these have many overtones and will present the fundamental frequency clearly.

With the illusion successfully modeled, the controls can be added. Firstly, a potentiometer is used to set the fundamental frequency. The same function as shown in Figure 5 is implemented so that the pot scrolls between notes of the western musical scale. Then, a potentiometer is used to adjust the threshold for the highpass filter. After this potentiometer is turned past a set position, the filtering begins. As it continues, more and more of the harmonics are filtered out, weakening the illusion. With this configuration, the demonstration shows the harmonic ranges where the fundamental is clearly heard even though it is not present. The code to achieve this is included in Figure 7.

```
int mode = analogRead(FILTER_CONTROL); // pot vlaue to control severity of filtration
float freq = getNoteFrequency(analogRead(NOTE_CONTROL)); // pot value to determine note
int harmonicMult; // number to dictate level of filtration

fundamental.begin(0.45, freq, WAVEFORM_TRIANGLE); // define the fundamental tone

switch (mode) {
  // control filtration
  case 0 ... 509:
    // play unfiltered tone (fundametal plays)
    playNormalTone();
    break;

  case 510 ... 1023:
    // play filtered signals (fundamental not played but is heard)
    harmonicMult = map(mode, 510, 1023, 1, 5); // when 1, onlt the fundamental is filtered
    // when 5, the fund and first 4 harmonics are filtered
    highpass.setHighpass(0, (float)harmonicMult * freq + 50.0, 0.7); // highpass filter
    playMissingFundamental(); // play filtered tone
    break;
}
```

Figure 7: Code to filter fundamental frequency to illustrate the missing fundamental illusion.

To continuously filter the harmonics as the potentiometer is rotated, the analog reading is mapped from 1 to 5. When the value is 1, it will only filter the fundamental, when it is 5, it will filter the fundamental and the first 4 harmonics. This is achieved by simply multiplying the mapped value by the fundamental and adding a slight buffer to set as the highpass cutoff. Because all harmonics are n multiples of the fundamental, this approach works well.

The connection scheme relevant to this control is shown in Table 3.

Teensy Pin	Connection	Function
A1	Potentiometer wiper	Set note
A2	Potentiometer wiper	Filter threshold

Table 3: Connection scheme for the missing fundamental illusion.

Shepard Tone

The final illusion demonstrated by this project is the Shepard tone. To generate this tone, several sine waves must be initialized and mixed to play through the master mixer. For this application, eight individual waves worked well. To start the illusion, each wave is assigned a frequency an octave apart from A1 to A8. To do so, a pointer array of the eight oscillator objects is constructed to allow for iteration over each wave, as hard coding the operations once per wave would be redundant. Traditional arrays of oscillator objects do not behave well, as pointed out by ChatGPT, which suggested implementing pointers instead. The code assigning the waves their initial frequencies is shown in Figure 8.

```
for (int i = 0; i < NUM_OSC; i++) {
    // initial octave positioning
    osc[i]->frequency(BASE_FREQ * pow(2.0, i));
    osc[i]->amplitude(0.0);
}
```

Figure 8: For loop used to iterate over the pointer array of oscillator objects and assign them their initial frequencies.

With this setup complete, the illusion can be generated. This works by equally raising the pitch of each wave so that the tone naturally climbs. It is important that each wave stays within the desired frequency range, between A0 and A8. Figure 9 includes the logic for constraining the waves within this range.

```
float octavePosition(float x) {
    /* Keeps all sine waves between 1st and 8th octave of base tone */

    while (x >= NUM_OSC) x -= NUM_OSC;
    while (x < 0) x += NUM_OSC;
    return x;
}
```

Figure 9: Code to constrain the frequency of the waves within the desired octave ranges.

The variable passed into this function for x represents the wave's phase, this keeps track of the octave positioning of each oscillator. The phase variable handles the octave shift of each wave. Based on a potentiometer reading, it is either incremented up by a large or small value, allowing for speed control of the rising pitch. Figure 10 outlines how this is implemented.

```
int speedPot = analogRead(POT_SPEED); // read pot for speed control
float speed = map(speedPot, 0, 1023, 1, 40) / 10000.0; // map to a speed value
// this vlaue determines how quickly the waves rise in pitch
phase += speed;
if (phase >= 1.0) {
    phase -= 1.0;
}

for (int i = 0; i < NUM_OSC; i++) {
    // iterate over all waves from the pointer array

    float pos = octavePosition(i + phase); // get octave position
    float freq = BASE_FREQ * pow(2.0, pos); // get the frequency based on octave position
```

Figure 10: Code implementing speed control and phase to define frequency of each wave.

Because the movement through each octave is identical, phase is held between 0 and 1. Now, the *octavePosition* function can be used to determine where each wave is in the range. Then, knowing that octaves are N multiples of the fundamental, a frequency can be defined for the oscillator.

While keeping the frequencies in this specific range is important, the key to the illusion is a bell curve gain distribution. This means that the lowest frequencies, around A0, and the highest frequencies, around A8, are set to be nearly silent, with gain rising exponentially towards the center of the frequency range. This means that the listener will not hear the transition from high frequency to low frequency, cementing the illusion of constantly rising pitch. The equation to create this gain distribution proved to be difficult to tune so it is constructed using the assistance of ChatGPT and is shown in Figure 11.

```

float bellGain(float pos) {
    /* Keeps middle frequencies loud and lower and upper frequencies quiet */
    /* Allows for seamless transition when waves reach their max octave and reset */

    float center = (NUM_OSC - 1) / 2.0; // finder center of frequency ranges across all octaves
    float width = 2.1; // width of loud region from the center

    float diff = pos - center; // used to determine how loud based on octave position
    float gain = exp(-(diff * diff) / (2.0 * width * width)); // bell curve gain determination

    return gain;
}

```

Figure 11: Function to set the bell curve gain distribution based on octave positioning of each oscillator.

This function is implemented similarly to *octavePosition*. The logic to set each oscillator's gain and frequency is shown in Figure 12.

```

for (int i = 0; i < NUM_OSC; i++) {
    // iterate over all waves from the pointer array

    float pos = octavePosition(i + phase); // get octave position
    float freq = BASE_FREQ * pow(2.0, pos); // get the frequency based on octave position

    while (freq > BASE_FREQ * pow(2.0, NUM_OSC)) {
        // if the wave's frequency is an octave too high, move it down an octave
        freq /= 2.0;
    }

    float gain = bellGain(pos) * 0.18; // determine gain based on octave position
    // 0.18 keeps it in a comfortable audio ranges

    osc[i] -> frequency(freq); // set the wave's frequency
    osc[i] -> amplitude(gain); // set the waves amplitude
}

```

Figure 12: Setting the gain and frequency for each wave.

With that, the Shepard tone is produced. The connection scheme for the speed control is shown in Table 4.

Teensy Pin	Connection	Function
A1	Potentiometer wiper	Set rising speed

Table 4: Connection scheme for Shepard tone control.

Combining Systems

To begin combining all of the audio illusions on the Teensy, the first step is to connect it to its audio shield. This board is conveniently equipped with an amplifier, digital to audio converter, and output headphone jack. The board can be seen in Figure 13.

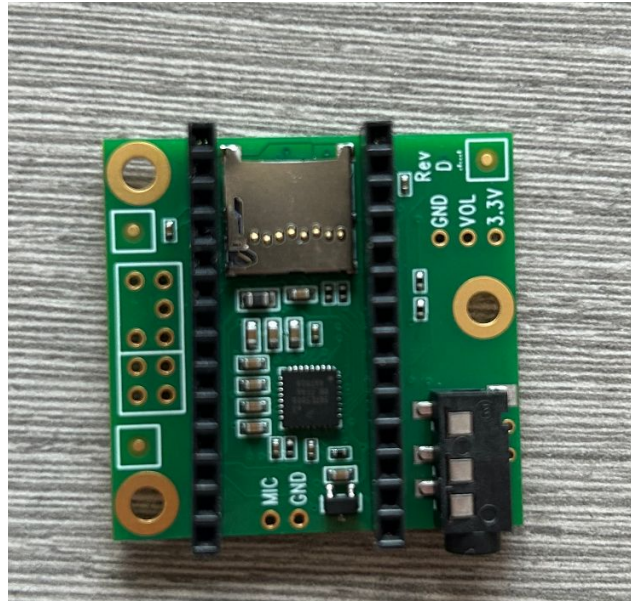


Figure 13: Teensy audio shield with socket headers soldered on.

Using the socket headers, the Teensy can be directly stacked onto the shield. This is convenient for testing the Audio library and successfully creating sounds with the device. However, to implement analog control as laid out above, this setup is not ideal as all of the Teensy's pins are occupied by the shield.

Fortunately, there are several analog pins that are not used by the shield, so the specific pins needed by the board can be wired to it directly leaving the rest of the Teensy's pins free. There is limited documentation online about which pins are required by the shield so ChatGPT provided a pinout that is useful for wiring them directly (see Table 4). With the Teensy and shield wired together, four potentiometers are implemented to control the device. Figure 14 shows the full device and Table 5 shows the connection scheme.

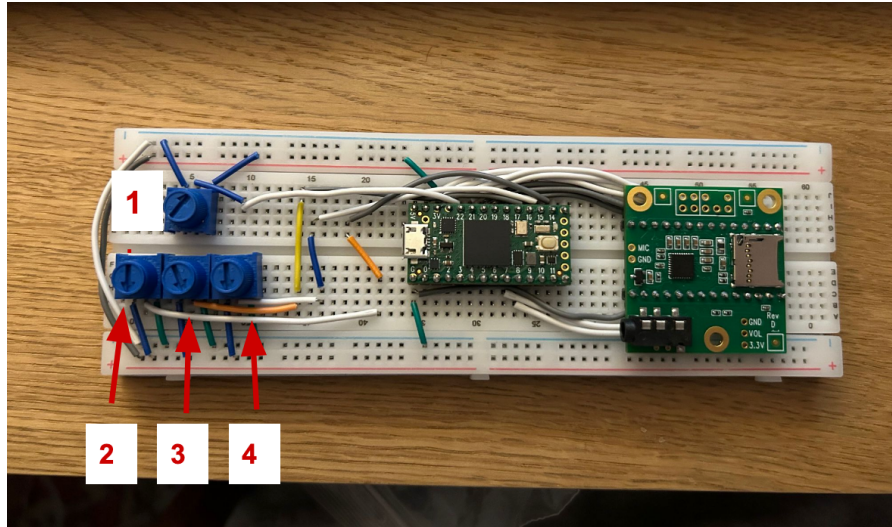


Figure 14: Full build with Teensy4.0, audio shield, and four potentiometers. The labels are useful for Table 5.

Teensy Pin	Connection	Function
A0	Potentiometer 2 wiper	Set volume
A1	Potentiometer 4 wiper	Set note
A2	Potentiometer 3 wiper	Filter threshold
A8	Potentiometer 1 wiper	Set outputted Illusion
23	Shield pin 23	Master clock
21	Shield pin 21	Bit clock for 12S
20	Shield pin 20	Left Right Audio Clock
7	Shield pin 7	Headphone out connection
8	Shield pin 8	Mic in connection
18	Shield pin 18	I2C Data
16	Shield pin 19	I2C Clock

Table 5: Comprehensive connection scheme for the entire device.

With the physical setup complete, the logic to link all illusion scripts and control which illusion is playing is implemented. This is done in the main *audio_illusions.ino* with a simple switch case shown in Figure 15.

```
void loop() {
  /* LOOP */
  volumeControl(); // update output volume (see audio_setup.cpp)

  // 10 bit pot value to switch between illusion modes
  int control = analogRead(CONTROL_PIN);

  switch(control) {
    /* Control which illusion is playing */
    case 200 ... 400:
      // demonstrate binaural beats
      binauralBeats();
      break;

    case 500 ... 700:
      // demonstrate missing fundamental (see missing_fundamental.cpp)
      demonstrateMF();
      break;

    case 800 ... 1000:
      // play shepard tone (see shepard.cpp)
      shepard();
      break;

    default:
      // off by default
      stopShepard();
      stopMF();
      stopBinaural();
      break;
  }
}
```

Figure 15: Loop function in the main *.ino* file run continuously by the Teensy.

Essentially, the Teensy reads the illusion control potentiometer (potentiometer 1) and plays the illusion corresponding to the definition in the switch case. It should be noted that in each script corresponding to an illusion there is a function to shut them off, either by zeroing wave amplitude or mixer gain. Each of these functions is called by default in the switch case. Because the thresholds defined in the switch case have space between them, no two illusions will activate simultaneously. This approach is highly effective in switching between illusions. It should also be noted that at the beginning of *loop()* the *volumeControl()* function is called, this allows for highly responsive dynamic audio level adjustments while the program is operating.

Conclusions - Performance and Next Steps

The project demonstrated successful generation of the audio illusions using a Teensy 4.0 and Teensy Audio Shield. The binaural beats, missing fundamental and Shepard tone all were able to be produced using this hardware, creating convincing audio illusions with one general output.

The final product created these illusions reliably. The audio library was able to handle multiple oscillators seamlessly. The binaural beat was able to consistently produce varying frequencies from the difference of the two frequencies. The missing fundamental produced convincing results using the audio filter, helping to isolate the tones to give a clear demonstration of the illusion. Finally, the Shepard tone was able to successfully generate a tone that seemingly endlessly rose in frequency. The hardware (potentiometers) provided a great way for real-time interaction with these illusions as they were generated,

In order to push this project further next steps such as adding new audio illusions could be taken. Some examples of illusions that may be added are Deutsch's scale or the Glissando illusion, which could be implemented by adding more hardware such as multiple speakers. Along with that, current illusions could be improved. For example, enhancing the quality of the transition for the Shepard tone and making the illusion more seamless.

Overall the project was a success and has provided room to add and explore many more auditory illusions using the Teensy microcontroller.

(Full code is located in GitHub)

References

Campo, Joseph. "Understanding the Missing Fundamental Phenomenon." *TeaHouse Studios, LLC*, www.teahousestudios.org/blog-posts/understanding-the-missing-fundamental-phenomenon. Accessed 5 May 2026.

Ingendoh, Ruth Maria, et al. "Binaural Beats to Entrain the Brain? A Systematic Review of the Effects of Binaural Beat Stimulation on Brain Oscillatory Activity, and the Implications for Psychological Research and Intervention." *PloS One*, U.S. National Library of Medicine, 19 May 2023, pmc.ncbi.nlm.nih.gov/articles/PMC10198548/#sec001.

Splice. "The Shepard Tone: What It Is and How It Works." Splice Blog, splice.com/blog/how-shepard-tone-works/. Accessed 5 May 2026.